

# Clínica Dental

## Sistema de Gestión para Clínica Dental

Alejandro Páez Milán (1º DAW)

Proyecto Java JPA con interfaz gráfica Swing

Mayo 2026

## Índice

---

- [1. Introducción](#)
- [2. Estructura del Proyecto](#)
- [3. Base de Datos](#)
- [4. Entidades JPA \(Modelos\)](#)
- [5. Controladores](#)
- [6. Vistas \(GUI Swing\)](#)
- [7. Configuración del Proyecto](#)
- [8. Cómo Ejecutar](#)

## 1. Introducción

---

Aplicación Java de escritorio para la gestión de una clínica dental. Permite administrar pacientes, doctores, citas, tratamientos e historiales clínicos mediante una interfaz gráfica Swing.

### Tecnologías utilizadas

| Tecnología | Versión | Propósito |
|------------|---------|-----------|
|------------|---------|-----------|

|                           |          |                                                       |
|---------------------------|----------|-------------------------------------------------------|
| Java                      | 25 (LTS) | Lenguaje de programación principal                    |
| Jakarta Persistence (JPA) | 2.2      | Mapeo objeto-relacional (ORM)                         |
| EclipseLink               | 2.7.12   | Proveedor JPA (implementación)                        |
| MySQL                     | 8.0      | Base de datos relacional                              |
| MySQL Connector           | 8.0.33   | Conector JDBC para MySQL                              |
| Java Swing                | -        | Biblioteca gráfica para la interfaz de usuario        |
| Maven                     | -        | Herramienta de construcción y gestión de dependencias |

### Funcionalidades principales

- Gestión completa de pacientes (alta, modificación, baja)
- Gestión de doctores con especialidades
- Creación y gestión de citas (asociando paciente y doctor)
- Catálogo de tratamientos dentales
- Historial clínico por paciente
- Relación N:M entre citas y tratamientos (Cita\_Tratamiento)

## 2. Estructura del Proyecto

---

El proyecto sigue una arquitectura por capas:

| Capa        | Paquete                                | Responsabilidad                                  |
|-------------|----------------------------------------|--------------------------------------------------|
| Modelo      | <code>models.modelos.entidades</code>  | Entidades JPA que mapean las tablas de la BD     |
| Controlador | <code>controllers.controladores</code> | Lógica CRUD, comunicación con la BD mediante JPA |
| Vista       | <code>views</code>                     | Interfaz gráfica Swing con paneles y diálogos    |
| Utilidad    | <code>util</code>                      | Clases auxiliares (SharedEntityManagerFactory)   |
| Entrada     | <code>daw</code>                       | Clase principal Main.java                        |

## Árbol de directorios

```

clinica_dental/
├── pom.xml                                # Configuración
├── ClinicaDental.sql                      # Script de
│   creación de la BD
├── migracion_cita_tratamiento.sql         # Migración para
│   añadir columna id
├── src/main/java/
│   ├── daw/Main.java                     # Punto de entrada
│   └── models/modelos/entidades/        # Entidades JPA (6
│       clases)
│       ├── Paciente.java
│       ├── HistorialClinico.java
│       ├── Doctor.java
│       ├── Cita.java
│       ├── Tratamiento.java
│       └── CitaTratamiento.java
│   └── controllers/controladores/        # Controladores
│       CRUD (6 clases)
│       └── PacienteController.java

```

```

    HistorialClinicoController.java
    DoctorController.java
    CitaController.java
    TratamientoController.java
    CitaTratamientoController.java

    views/                                # Paneles Swing (6
clases)                                # Ventana principal
    MainFrame.java
con pestañas
    PacientePanel.java
    DoctorPanel.java
    CitaPanel.java
    TratamientoPanel.java
    HistorialClinicoPanel.java
    CitaTratamientoPanel.java

    util/
        SharedEntityManagerFactory.java

src/main/resources/META-INF/
    persistence.xml                    # Configuración
JPA

```

## 3. Base de Datos

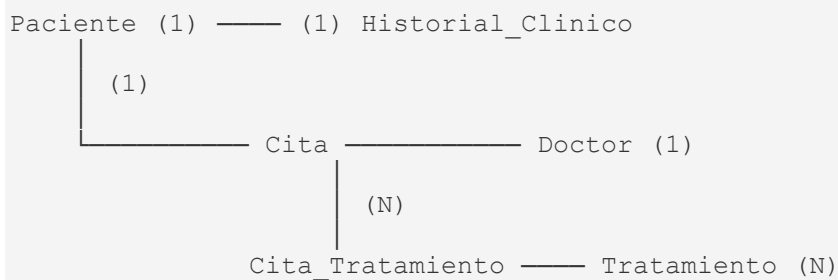
### Diseño relacional

La base de datos `clinica_dental` consta de 6 tablas:

| Tabla                    | PK           | FK                     | Descripción                         |
|--------------------------|--------------|------------------------|-------------------------------------|
| <b>Paciente</b>          | codPaciente  | -                      | Datos personales del paciente       |
| <b>Historial_Clinico</b> | codHistorial | codPaciente → Paciente | Historial médico (1:1 con Paciente) |
| <b>Doctor</b>            | codDoctor    | -                      | Datos del doctor/odontólogo         |

|                         |                |                            |                                         |
|-------------------------|----------------|----------------------------|-----------------------------------------|
| <b>Cita</b>             | codCita        | codPaciente,<br>codDoctor  | Citas programadas                       |
| <b>Tratamiento</b>      | codTratamiento | -                          | Catálogo de<br>tratamientos<br>dentales |
| <b>Cita_Tratamiento</b> | id             | codCita,<br>codTratamiento | Relación N:M<br>(puente)                |

## Diagrama de relaciones



## Script de creación

El archivo `ClinicaDental.sql` contiene las sentencias CREATE TABLE con:

- Claves primarias autoincrementales
- Claves foráneas con RESTRICT y CASCADE
- Constraints CHECK para validación de datos
- UNIQUE KEY en combinaciones que deben ser únicas

**Nota:** La tabla Cita\_Tratamiento utiliza una PK simple (columna `id` AUTO\_INCREMENT) en lugar de la PK compuesta (`codCita`, `codTratamiento`). Esto sigue la directriz JPA de usar `@Id` `@GeneratedValue`. La unicidad de la combinación (`codCita`, `codTratamiento`) se mantiene mediante UNIQUE KEY.

## 4. Entidades JPA (Modelos)

Todas las entidades están en el paquete `models.modelos.entidades`. Utilizan anotaciones JPA estándar (`@Entity`, `@Table`, `@Id`, `@GeneratedValue`, `@Column`, `@ManyToOne`, `@OneToOne`, `@OneToMany`, `@NamedQueries`) y están comentadas en español con explicaciones didácticas.

### Paciente

| Campo           | Tipo    | Columna BD              | Descripción                     |
|-----------------|---------|-------------------------|---------------------------------|
| codPaciente     | Integer | codPaciente (PK, AI)    | Identificador único             |
| dni             | String  | DNI (UNIQUE, NOT NULL)  | Documento nacional de identidad |
| nombre          | String  | Nombre (NOT NULL)       | Nombre del paciente             |
| apellidos       | String  | Apellidos (NOT NULL)    | Apellidos del paciente          |
| fechaNacimiento | Date    | Fecha_Nacimiento (DATE) | Fecha de nacimiento             |
| telefono        | String  | telefono                | Teléfono de contacto            |
| email           | String  | email                   | Correo electrónico              |

|           |        |           |                  |
|-----------|--------|-----------|------------------|
| direccion | String | direccion | Dirección postal |
|-----------|--------|-----------|------------------|

**Relaciones:**

- `@OneToOne(mappedBy = "paciente")` → HistorialClinico (1:1, lado inverso)
- `@OneToMany(mappedBy = "paciente")` → Cita (1:N)

**Named Queries:** findAll, findById, findByDNI, findByNombre, findByApellidos

**HistorialClinico**

| Campo                | Tipo     | Columna BD               | Descripción           |
|----------------------|----------|--------------------------|-----------------------|
| codHistorial         | Integer  | codHistorial (PK, AI)    | Identificador único   |
| paciente             | Paciente | codPaciente (FK, UNIQUE) | Paciente asociado     |
| alergias             | String   | Alergias                 | Alergias conocidas    |
| enfermedadesCronicas | String   | EnfermedadesCronicas     | Enfermedades crónicas |
| grupoSanguineo       | String   | GrupoSanguineo           | Grupo sanguíneo       |

|                        |        |                                   |                   |
|------------------------|--------|-----------------------------------|-------------------|
| observacionesGenerales | String | ObservacionesGenerales (TEXT)     | Observaciones     |
| fechaAlta              | Date   | FechaAlta (DATETIME, DEFAULT NOW) | Fecha de creación |

**Relaciones:**

- `@OneToOne(optional = false) @JoinColumn(name = "codPaciente")` → Paciente (1:1, lado propietario)

**Named Queries:** findAll, findById, findByPaciente

**Doctor**

| Campo            | Tipo    | Columna BD               | Descripción               |
|------------------|---------|--------------------------|---------------------------|
| codDoctor        | Integer | codDoctor (PK, AI)       | Identificador único       |
| numeroColegiado  | String  | NumeroColegiado (UNIQUE) | Número de colegiado       |
| nombre           | String  | Nombre (NOT NULL)        | Nombre del doctor         |
| especialidad     | String  | Especialidad (NOT NULL)  | Especialidad odontológica |
| telefonoContacto | String  | TelefonoContacto         | Teléfono de contacto      |



Relaciones:

- @OneToMany(mappedBy = "doctor") → Cita (1:N)

Named Queries: findAll, findById, findByNombre, findByEspecialidad

Cita

| Campo      | Tipo     | Columna BD        | Descripción                     |
|------------|----------|-------------------|---------------------------------|
| codCita    | Integer  | codCita (PK, AI)  | Identificador único             |
| paciente   | Paciente | codPaciente (FK)  | Paciente de la cita             |
| doctor     | Doctor   | codDoctor (FK)    | Doctor que atiende              |
| fecha      | Date     | fecha (DATE)      | Fecha de la cita                |
| horaInicio | Date     | horaInicio (TIME) | Hora de inicio                  |
| horaFin    | Date     | horaFin (TIME)    | Hora de fin                     |
| estado     | String   | estado (NOT NULL) | Programada/Completada/Cancelada |

|               |      |                          |                                |
|---------------|------|--------------------------|--------------------------------|
| fechaCreacion | Date | fechaCreacion (DATETIME) | Fecha de creación del registro |
|---------------|------|--------------------------|--------------------------------|

**Relaciones:**

- `@ManyToOne(optional = false) @JoinColumn(name = "codPaciente")` → Paciente (N:1)
- `@ManyToOne(optional = false) @JoinColumn(name = "codDoctor")` → Doctor (N:1)
- `@OneToMany(mappedBy = "cita")` → CitaTratamiento (1:N)

**Named Queries:** `findAll`, `findById`, `findByFecha`, `findByEstado`

**Tratamiento**

| Campo             | Tipo    | Columna BD                   | Descripción            |
|-------------------|---------|------------------------------|------------------------|
| codTratamiento    | Integer | codTratamiento (PK, AI)      | Identificador único    |
| nombreTratamiento | String  | NombreTratamiento (NOT NULL) | Nombre del tratamiento |
| descripcion       | String  | Descripcion (TEXT)           | Descripción detallada  |
| precioEstimado    | Double  | PrecioEstimado (DECIMAL)     | Precio del tratamiento |
| duracionMinutos   | Integer | DuracionMinutos (NOT NULL)   | Duración en minutos    |

**Relaciones:**

- `@OneToMany(mappedBy = "tratamiento")` → CitaTratamiento (1:N)

**Named Queries:** findAll, findById, findByName

## CitaTratamiento

| Campo         | Tipo        | Columna BD               | Descripción                     |
|---------------|-------------|--------------------------|---------------------------------|
| id            | Integer     | id (PK, AI)              | Identificador único del detalle |
| cita          | Cita        | codCita (FK)             | Cita asociada                   |
| tratamiento   | Tratamiento | codTratamiento (FK)      | Tratamiento asociado            |
| cantidad      | Integer     | Cantidad (DEFAULT 1)     | Cantidad de unidades            |
| fechaRegistro | Date        | FechaRegistro (DATETIME) | Momento del registro            |

## Relaciones:

- `@ManyToOne(optional = false) @JoinColumn(name = "codCita")` → Cita (N:1)
- `@ManyToOne(optional = false) @JoinColumn(name = "codTratamiento")` → Tratamiento (N:1)

**Named Queries:** findAll, findById, findByCita, findByTratamiento

**Nota sobre CitaTratamiento:** Es la tabla puente para la relación N:M entre Cita y Tratamiento. Siguiendo la directriz JPA, tiene su propio `@Id @GeneratedValue` en lugar de usar `@IdClass` con clave

compuesta. La BD mantiene la unicidad mediante UNIQUE KEY sobre (codCita, codTratamiento).

## 5. Controladores

Los controladores están en `controllers.controladores` y siguen el patrón CRUD estándar. Cada controlador gestiona una entidad JPA y proporciona métodos para crear, leer, actualizar y eliminar registros.

### Estructura común de todos los controladores

Todos los controladores comparten la misma estructura:

- **Constructor:** obtiene el EntityManagerFactory de `SharedEntityManagerFactory.getInstance()`
- **getEntityManager():** método privado que crea un nuevo EntityManager para cada operación
- **create(entidad):** inicia transacción, llama a `em.persist()`, hace commit
- **findById(id):** busca por clave primaria con `em.find()`
- **findAll():** ejecuta una NamedQuery para obtener todos los registros
- **update(entidad):** sincroniza con `em.merge()`
- **delete(id):** busca y elimina con `em.remove()`
- **deleteAll():** borra todo con SQL nativo y reinicia AUTO\_INCREMENT

### Patrón de transacciones

```
EntityManager em = getEntityManager();
EntityTransaction tx = em.getTransaction();
try {
    tx.begin();
    em.persist(entidad);    // o em.merge(), em.remove()
    tx.commit();
} catch (Exception ex) {
    if (tx.isActive()) { tx.rollback(); }
    throw new RuntimeException("Error", ex);
} finally {
    em.close();
}
```

## SharedEntityManagerFactory

Clase en `util.SharedEntityManagerFactory` que implementa un singleton thread-safe de `EntityManagerFactory`. Todos los controladores comparten la misma instancia, evitando crear múltiples conexiones a la base de datos innecesariamente. Incluye un shutdown hook para cerrar la fábrica al terminar la aplicación.

### Listado de controladores y sus métodos

| Controlador                | create() | findById() | findAll() | update() | delete() |
|----------------------------|----------|------------|-----------|----------|----------|
| PacienteController         | ✓        | ✓          | ✓         | ✓        | ✓        |
| HistorialClinicoController | ✓        | ✓          | ✓         | ✓        | ✓        |
| DoctorController           | ✓        | ✓          | ✓         | ✓        | ✓        |
| CitaController             | ✓        | ✓          | ✓         | ✓        | ✓        |
| TratamientoController      | ✓        | ✓          | ✓         | ✓        | ✓        |
| CitaTratamientoController  | ✓        | ✓          | ✓         | ✓        | ✓        |

**Importante:** Los métodos `deleteAll()` utilizan SQL nativo para borrar registros y reiniciar `AUTO_INCREMENT`. El orden de borrado es

importante para respetar las claves foráneas. Además, referencian explícitamente la base de datos `clinica_dental.` como prefijo.

## 6. Vistas (GUI Swing)

---

Todas las vistas están en el paquete `views`. Utilizan Java Swing con `GridBagLayout` para los formularios y `JTable` con `DefaultTableModel` para mostrar los datos.

### MainFrame

Ventana principal (`JFrame`) que contiene un `JTabbedPane` con 6 pestañas:

1. **Pacientes**
2. **Doctores**
3. **Citas**
4. **Tratamientos**
5. **Historial Clínico**
6. **Cita-Tratamiento**

Incluye un botón "Salir" en la parte inferior. Se inicia desde el Event Dispatch Thread mediante `SwingUtilities.invokeLater()`.

### Patrón común de los paneles

Cada panel sigue esta estructura:

- Constructor: crea el controlador, inicializa componentes, carga datos
- **initComponents()**: configura la tabla (no editable, selección única) y los botones (Nuevo, Editar, Eliminar)
- **loadData()**: vacía la tabla, obtiene datos del controlador, rellena filas
- **mostrarDialogo(entidad)**: abre un `JDialog` modal con formulario (crear o editar)
- **editar()**: obtiene la fila seleccionada y abre el diálogo con datos precargados
- **eliminar()**: confirma y borra el registro seleccionado

## **Detalle de cada panel**

### **PacientePanel**

Columnas: ID, DNI, Nombre, Apellidos, Fecha Nac., Teléfono, Email, Dirección. Diálogo con 7 campos de texto y formato de fecha dd/MM/yyyy.

### **DoctorPanel**

Columnas: ID, N° Colegiado, Nombre, Especialidad, Teléfono. Diálogo con JComboBox de especialidades válidas (Odontología General, Ortodoncia, etc.).

### **CitaPanel**

Columnas: ID, Paciente, Doctor, Fecha, Hora Inicio, Hora Fin, Estado. Diálogo con JComboBox de pacientes y doctores (cargados de la BD), campos de fecha y hora, y JComboBox de estado (Programada, Completada, Cancelada).

### **TratamientoPanel**

Columnas: ID, Nombre, Descripción, Precio (€), Duración (min). Diálogo con campos para nombre, descripción, precio y duración.

### **HistorialClinicoPanel**

Columnas: ID, Paciente, Alergias, Enfermedades, Grupo Sang., Fecha Alta. Diálogo con JComboBox de pacientes, JComboBox de grupo sanguíneo (A+, A-, B+, etc.), JTextArea para observaciones. El paciente se bloquea en edición. Valida que el paciente no tenga ya un historial.

### **CitaTratamientoPanel**

Columnas: ID, Cita, Tratamiento, Cantidad, Fecha Registro. Diálogo con JComboBox de citas (paciente + fecha) y tratamientos (nombre + precio). Valida que no exista duplicado (misma cita + mismo tratamiento).

## **Manejo de errores en la GUI**

Todos los paneles capturan excepciones en las operaciones CRUD y muestran mensajes de error mediante

`JOptionPane.showMessageDialog()`. Las validaciones incluyen:

- Formato de fecha incorrecto
- Valores numéricos inválidos
- Selección obligatoria de fila antes de editar/eliminar
- Confirmación antes de eliminar

## 7. Configuración del Proyecto

---

### **pom.xml**

Archivo Maven con las siguientes dependencias:

```
<dependencies>
  <!-- MySQL Connector -->
  <dependency>
    <groupId>mysql</groupId>
    <artifactId>mysql-connector-java</artifactId>
    <version>8.0.33</version>
  </dependency>

  <!-- EclipseLink (proveedor JPA) -->
  <dependency>
    <groupId>org.eclipse.persistence</groupId>
    <artifactId>org.eclipse.persistence.jpa</artifactId>
    <version>2.7.12</version>
  </dependency>
  <!-- Otras dependencias de EclipseLink: core, asm,
antlr, moxy -->
  <dependency>
    <groupId>org.eclipse.persistence</groupId>
    <artifactId>jakarta.persistence</artifactId>
    <version>2.2.3</version>
  </dependency>
</dependencies>

<properties>
  <maven.compiler.release>25</maven.compiler.release>
  <exec.mainClass>daw.Main</exec.mainClass>
</properties>
```

### **persistence.xml**

Configuración JPA en `src/main/resources/META-INF/persistence.xml`:



```

<persistence-unit name="clinica_dental" transaction-
type="RESOURCE_LOCAL">

<provider>org.eclipse.persistence.jpa.PersistenceProvider</p
rovider>
  <class>models.modelos.entidades.Paciente</class>
  <class>models.modelos.entidades.Doctor</class>
  <class>models.modelos.entidades.Cita</class>
  <class>models.modelos.entidades.Tratamiento</class>
  <class>models.modelos.entidades.CitaTratamiento</class>
  <class>models.modelos.entidades.HistorialClinico</class>

  <properties>
    <property name="javax.persistence.jdbc.url"
value="jdbc:mysql://localhost:3306/clinica_dental?serverTime
zone=UTC"/>
    <property name="javax.persistence.jdbc.user"
value="root"/>
    <property name="javax.persistence.jdbc.driver"
value="com.mysql.cj.jdbc.Driver"/>
    <property name="javax.persistence.jdbc.password"
value=""/>
    <property name="eclipselink.logging.level"
value="OFF"/>
  </properties>
</persistence-unit>

```

**Nota:** La contraseña de MySQL está vacía (root sin contraseña). Si tu MySQL tiene contraseña, cámbiala en el persistence.xml.

## 8. Cómo Ejecutar

### Requisitos previos

- Java 25 (JDK instalado)
- MySQL 8.0+ ejecutándose en localhost:3306
- Maven (integrado en VS Code o instalado por separado)

### Pasos para ejecutar

1. **Crear la base de datos:** ejecutar `ClinicaDental.sql` en MySQL
2. **Migrar Cita\_Tratamiento** (si tu BD tiene PK compuesta): ejecutar `migracion_cita_tratamiento.sql`
3. **Abrir el proyecto** en VS Code
4. **Compilar:** VS Code compila automáticamente al guardar, o desde terminal con `mvn compile`

5. **Ejecutar:** desde VS Code, abrir Main.java y presionar F5 (Run), o ejecutar `daw.Main`

## Acceso a la base de datos

| Parámetro  | Valor                                                                      |
|------------|----------------------------------------------------------------------------|
| URL        | <code>jdbc:mysql://localhost:3306/clinica_dental?serverTimezone=UTC</code> |
| Usuario    | <code>root</code>                                                          |
| Contraseña | (vacía)                                                                    |
| Puerto     | <code>3306</code>                                                          |

## Resumen de archivos del proyecto (21 archivos fuente)

| Tipo          | Cantidad | Archivos                                                                                                                           |
|---------------|----------|------------------------------------------------------------------------------------------------------------------------------------|
| Entidades JPA | 6        | Paciente, HistorialClinico, Doctor, Cita, Tratamiento, CitaTratamiento                                                             |
| Controladores | 6        | PacienteController, HistorialClinicoController, DoctorController, CitaController, TratamientoController, CitaTratamientoController |
| Vistas Swing  | 7        | MainFrame, PacientePanel, DoctorPanel, CitaPanel, TratamientoPanel, HistorialClinicoPanel, CitaTratamientoPanel                    |

|               |   |                                                      |
|---------------|---|------------------------------------------------------|
| Utilidades    | 1 | SharedEntityManagerFactory                           |
| Entrada       | 1 | Main                                                 |
| Configuración | 2 | pom.xml, persistence.xml                             |
| SQL           | 2 | ClinicaDental.sql,<br>migracion_cita_tratamiento.sql |